

Daffodil Power Management and Configuration

Daffodil firmware reference — config switch (CSW) function mapping, per-device CSW calibration, battery/solar power management, and LED status display. Last updated 2026-09-01 (evening revision — CSW calibration validated end-to-end, Factory tool automation, switch 0011 reassigned, LED display fixes).

1. Overview

Daffodil is configured by a 5-position DIP switch (the "config switch" or CSW), read as an analog voltage through a resistor ladder into the ADS1115 ADC. The first 4 switches select the device's **function** (which sensors it reads and what it reports); the 5th switch selects whether the device should run its **solar-aware power schedule**.

This document covers four related systems:

- 1. The switch → function mapping (what each of the 16 four-bit patterns does).
- 2. Why the raw ADC reading needs a one-time **per-device calibration** step, and how to run it (now automated by the Factory device-configuration tool).
- 3. How the firmware decides sleep timing, LED brightness, and battery protection — and how that now works correctly whether or not a unit is configured for solar.
- 4. How to read the status LED display.

2. Config Switch (CSW) Function Mapping

The switch has 5 positions. The first 4 select the function; the 5th selects solar power mode and does **not** change which function is selected — e.g. 00000 and 00001 are both FUN_1_FLOW , the only difference is whether the solar power schedule is active.

Bits (1234)	Function
0000	FUN_1_FLOW
1000	FUN_2_FLOW
0100	FUN_1_FLOW_1_TANK
1100	FUN_1_TANK
0010	FUN_2_TANK
1010	DAFFODIL_SCEPTIC_TANK
0110	DAFFODIL_WATER_TROUGH

1110	DAFFODIL_WATER_TROUGH_TANK1
0011	DAFFODIL_2_WATER_TROUGH
0001	unassigned (reserved)
1001	unassigned (reserved)
0101	unassigned (reserved)
1101	unassigned (reserved)
1011	unassigned (reserved)
0111	unassigned (reserved)
1111	unassigned (reserved)

5th bit (`usingSolarPower`): the function selected by bits 1–4 is identical either way — e.g. `00000` and `00001` are both `FUN_1_FLOW`. This bit is a sensor-reading-frequency vs. battery-duration tradeoff, not literally "solar attached or not": `1` defers to the solar-efficiency-based sleep schedule (sleeps more, samples less — for unattended solar-charged deployments that don't need frequent readings, e.g. a trough level sensor); `0` samples more often and doesn't defer to that schedule (e.g. continuous flow monitoring, at the cost of more battery use). Battery over-discharge protection applies either way — see §4.1.

Notes:

- `DAFFODIL_2_WATER_TROUGH` (`0011` , added 2026-09-01) is 2 independent troughs read via UART ultrasonic sensors (AJ-SR04M-style protocol: 9600 baud, `0xFF` header + 2 data bytes + checksum, on Serial1/pin18 and Serial2/pin33) instead of the single trigger/echo sonar the other trough modes use. The switch position and firmware constant exist; **sensor-reading and LED-display code is not yet implemented**, pending the sensor hardware arriving. `0011` used to be a duplicate `DAFFODIL_WATER_TROUGH` slot — reclaimed for this.
- 6 of the 16 four-bit patterns are still unassigned/reserved — the device just applies the solar-bit setting and otherwise does nothing for those.
- `VOLTAGE_MONITOR` , `DAFFODIL_TEMP_SOILMOISTURE` , and `DAFFODIL_LIGHT_DETECTOR` exist as function constants in firmware but are **not currently reachable** via any switch position — no wiring or assigned slot in the current 5-bit scheme.

3. Per-Device CSW Calibration

3.1 Why this is necessary

The switch ladder's pull-up resistor (R2) is fed by `DEVICE_POWER` , which is the `V50` rail passed through a load switch — **not** `V50_I` (the panel/USB input firmware measures on another ADC channel). `V50_I` only tracks `V50` when USB/panel power dominates; when the unit is running on battery, `V50` is generated independently by a boost converter (fed by the battery) and can differ from `V50_I` by a device-dependent amount. Since the ADS1115 measures an absolute voltage (not

radiometric to its own supply), **every raw CSW reading scales proportionally with whatever the actual V50 level is** — and that level varies from battery to battery (confirmed empirically: two different battery units showed a consistent ~13% difference in raw readings across every switch position).

Because there's no spare ADC channel to measure `V50` directly (all 4 ADS1115 channels are already used: `tank1`, `tank2`, `CSW`, `V50_I`), and no PCB modification is possible on deployed units, the fix is a one-time **per-device scale calibration** stored in flash (NVS), rather than a single fixed threshold table.

3.2 Calibration procedure

Run this once per device at install time (and again any time the battery is swapped for a substantially different one):

1. Set the DIP switches to `00000` (all off, no solar — doesn't drive the LEDs, safe to leave on battery for the reset this needs). **Double-check the switches physically before proceeding** — arming the calibration while the switches are at any other position captures the wrong reference and silently corrupts the whole table (confirmed 2026-09-01: this is the single most common way to get this step wrong).
2. Power on / reset and let the device boot normally.
3. Send the serial command `CalibrateCSWReference`. This does **not** read the switch itself — it only arms a flag in flash. (A command-time read would happen mid-runtime, with WiFi/LoRa already active — a different electrical load on `DEVICE_POWER / V50` than the real decode ever sees at early boot, which gives a wrong reference reading. Confirmed 2026-08-31.)
4. **Reset the device again** (switches still at `00000`). This boot captures the actual reference reading at the exact point in `setup()` where the real switch decode happens (before WiFi/LoRa start), and clears the arm flag.
5. Confirm via the `printCSWData` serial command — `cswReferenceRaw` should be a plausible `00000` reading (roughly 7000–8500 on the units tested so far), not `0` (never calibrated) and not matching some other switch position's raw value (a sign the switches moved before step 4).
6. Set the switches to the real desired function and reset again for normal operation.

Factory tool automation: as of 2026-09-01, `ConfigureDeviceProcessingHandler` (the `/home/ari/Data/DigitalStables/FactorySystem/factory` device-configuration flow) runs this procedure automatically as the last step after flashing product firmware — it prompts for the two switch/reset actions via the same status log the rest of device configuration already uses, sends `CalibrateCSWReference` and verifies `cswReferenceRaw` itself, and logs a clear warning if the capture didn't take. New units configured through the Factory tool get this for free; it only needs doing manually for units configured/flashed some other way.

3.3 How the calibration is applied

```
scaleFactor    = 8326 / cswReferenceRaw    (8326 = the original 2026-08-27
reference calibration's 00000 point)
cswDecodeValue = rawCSWValue * scaleFactor
```

`cswDecodeValue` (not the raw ADC reading) is what gets compared against the threshold table. If a device has never been calibrated (`cswReferenceRaw == 0`), `scaleFactor` defaults to `1.0` — behavior is unchanged from before this fix existed.

This was validated end-to-end on the bench unit on 2026-09-01: every one of the 16 solar-on switch positions (`00001` – `11111`) decoded to the correct function after calibration, each landing within single-digit ADC counts of the original reference table.

3.4 Relevant serial commands

Command	Description
<code>CalibrateCSWReference</code>	Arms per-device CSW calibration; requires a reset to actually capture it (see §3.2)
<code>printCSWData</code>	Prints <code>rawCSWValue</code> , <code>cswReferenceRaw</code> , <code>cswScaleFactor</code> , <code>cswDecodeValue</code> , <code>noBatteryDetected</code> , and the decoded function
<code>printCurrentDSDData</code>	Prints the full current sensor/runtime snapshot, including the CSW decode

4. Power Management

4.1 Two independent concepts

Earlier firmware conflated "does this unit have a battery worth protecting" with "is the solar bit set" — both were gated by the same `usingSolarPower` flag. This meant a unit configured without solar (`bit5 = 0`) got **no battery protection at all** (no COMMA-mode critical cutoff, no low-voltage sleep, no low-voltage LED/WiFi shutoff), because the code assumed "no solar" meant "wall/USB powered, unlimited power available." That assumption doesn't hold for a battery-only unit that simply isn't using the solar schedule.

As of 2026-09-01 these are two separate flags:

- **hasBattery** (`= !noBatteryDetected` , computed once at boot from the same battery-presence heuristic used for CSW decode) — gates *all* over-discharge protection: COMMA mode, the low-voltage forced-sleep check, the low-voltage LED cutoff, and the low-voltage WiFi cutoff. This applies **regardless of the solar bit** — any unit with a real battery attached gets protected.
- **usingSolarPower** (`bit5`) — now purely a sensor-reading-frequency vs. battery-duration tradeoff. `bit5 = 1` : defer to the solar-efficiency-based sleep schedule (sleeps more, samples

less, optimized for unattended solar-charged deployments — e.g. a water trough level sensor that doesn't need minute-by-minute readings at 3am). `bit5 = 0` : sample more frequently, don't defer to the solar schedule (e.g. continuous flow monitoring) — but the battery is still protected via `hasBattery` if one is attached.

4.2 LED brightness

LED brightness is now a function of **actual available power**, computed by `calculatePowerAwareLedBrightness()`, for every unit with a battery — regardless of the solar bit. Previously, `usingSolarPower = false` forced LEDs to constant full brightness (255) with no power awareness at all, which would drain a battery-only non-solar unit.

The calculation, in order:

1. **Darkness** — if the BH1750 light sensor reads genuine night/deep-shade, cap brightness at `nightLedBrightness` (30).
2. **Battery current headroom** — `batteryCurrent` follows the project's sign convention (positive = discharging, negative = charging). If the last reading showed net *discharge* even accounting for the LEDs' own draw, brightness drops to `dimLedBrightness` (20). If net *charging* (headroom exists), brightness scales smoothly between `dimLedBrightness` and 255 in proportion to how much of one full LED load ($\text{NUM_LEDS} \times \sim 20 \text{ mA}$) that headroom could cover.
 - This is deliberately a smoothed, one-cycle-delayed feedback loop, not an instant on/off snap at 0 mA — a hard threshold would oscillate (LEDs on → current goes positive → LEDs off → current goes negative → LEDs back on → ...).
3. **Low battery voltage** — hard floor at `dimLedBrightness` if `batteryVoltage < minimumWifiVoltage` (3.28V), regardless of the above.

4.3 Battery protection thresholds

Threshold	Value	Effect
<code>sleepingVoltage</code>	3.12V	Force deep sleep — cliff edge for LiFePO4
<code>commaVoltage</code>	2.80V	COMMA mode — critically low, skip all work, wait for the battery to recover before resuming normal operation
<code>minimumLEDVoltage</code>	3.18V	Turn off LEDs entirely
<code>minimumWifiVoltage</code>	3.28V	Turn off WiFi (preserve power for LoRa)
<code>minimumInitWifiVoltage</code>	3.35V	Battery must sustain this for 30s before WiFi is allowed to start

All of the above now apply whenever `hasBattery` is true, independent of the solar bit (§4.1).

4.4 What stays solar-only

The following remain gated on `usingSolarPower` (`bit5 = 1`) specifically, since they depend on an actual solar/panel signal or represent the solar-specific sleep-scheduling policy, not battery protection in general:

- Solar-efficiency-based sleep scheduling (`PowerManager.calculateOptimalSleepTime`)
- Cloudy-day detection (`OPERATING_STATUS_CLOUDY`) and its sleep-time extension
- The efficiency-gated LED on/off decision (whether LEDs get any power at all this cycle) — brightness itself (§4.2) is solar-independent, but *whether* they're lit follows the solar schedule when `bit5 = 1`

5. LED Status Display

The status LEDs are a 3-row × 5-column grid (15 WS2812 LEDs), wired in plain row-major order — **not** serpentine — confirmed 2026-09-01 on the bench unit:

```
row1:  0  1  2  3  4
row2:  5  6  7  8  9
row3: 10 11 12 13 14
```

The display cycles through several states every ~2 seconds each (temperature, tank/trough/flow, internet status, LoRa TX, error/battery status). Within the tank/trough/flow state, two-slot modes (`FUN_2_FLOW` , `FUN_2_TANK` , `FUN_1_FLOW_1_TANK` , `DAFFODIL_WATER_TROUGH_TANK1`) show sensor 1 and sensor 2 back-to-back (two consecutive ~2s ticks) before the display moves on to the other states, rather than alternating once per full outer cycle — so both sensors' status is visible in quick succession, not spread minutes apart.

For flow-family modes, an "F" glyph is drawn using LEDs `{0,1,2,5,6,10}` (top bar + shorter middle bar + left stem) — its own color (red/blue) reflects whether that sensor shows flow. A separate marker LED indicates *which* sensor slot is currently on screen: led4 (top-right corner) for slot 1, led14 (bottom-right corner) for slot 2 — both deliberately outside the F glyph's own pixels (and outside the tank "tower" display's pixels) so the marker can never overwrite the symbol's actual status color.

6. Known Limitations / Open Items

- The non-solar half of the switch space (`00000` – `11110` , `bit5=0`) has not yet been swept through the calibrated decode table the way the solar-on half was on 2026-09-01 — the mapping should be identical (same 4-bit function table, same scale factor), but hasn't been bench-verified position-by-position yet.
- A hardware-level backfeed exists from `V50` into the battery-sense node through the boost converter (U5, ETA1061V50S2G) when no battery is attached — this is why `noBatteryDetected` needed a delta-based/settling-aware check rather than a simple voltage

threshold. Not an issue for real deployed units (which always have a battery), but relevant if bench-testing without one.

- `VOLTAGE_MONITOR` , `DAFFODIL_TEMP_SOILMOISTURE` , `DAFFODIL_LIGHT_DETECTOR` are unreachable via the current switch scheme (§2) — flagged for the website team's function reference, not something this firmware change addresses.
- `DAFFODIL_2_WATER_TROUGH` (`0011`) has its switch position, firmware constant, and (pending hardware) UART protocol identified, but no sensor-reading or LED-display code yet — see §2's note.